FastVideo

Danyang Zhuo

Duke University

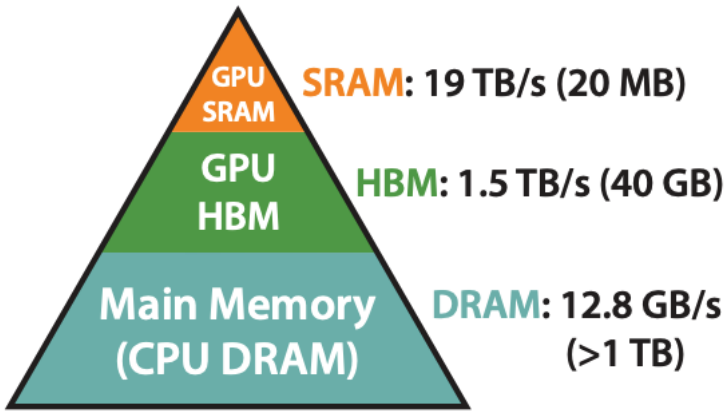
Session 1: kv cache management and optimization
1. Mingxing Liu - KV cache compression
2. Mrinal Goel - KV cache optimization
3. Yumo Fan, Yuhan Wei, Haoyuan Zheng - paged kvcomm
4. Michael Yuan - Stress testing KVCOMM
5. Zhuo Chen, Zhongming Zuo, Junyuan Deng - KV cache compression
6. Hemasai Suhas Kurapati - Failure surfaces in KV cache optimization

Session 2: Multi LoRA
1. Haiwei Yu, Shuai Wang - Adapter caching and prefetching in Punica
2. Jianjun Ou, Jiacheng Sang, Rongjian Wang - DeepResearch using Multi-LoRA training
3. Zhengyang Yu - Understanding multi-tenant lora inference
4. Manmit Singh, Ivan Rosales - LoRA for RL
5. Chongwen Zhao - Fine tune LoRA with Punica

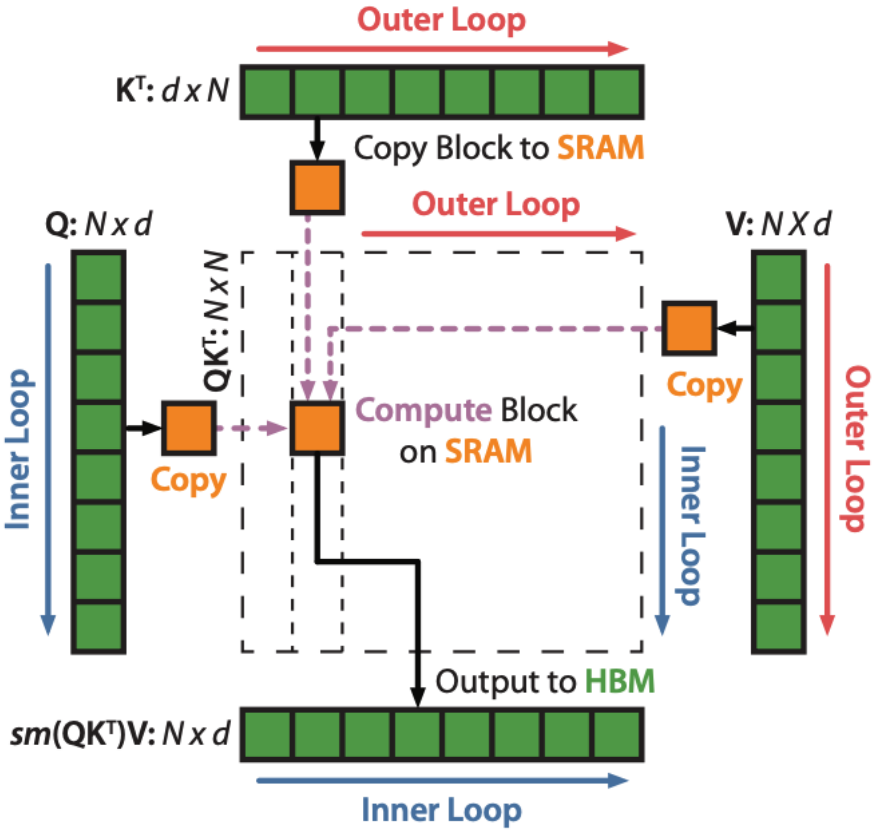
Session 3: Inference beyond GPUs
1. Lixun Zhang - minimal vLLM on apple silicon
2. Chunjiang Che, Ming-Yang Ho, Chenyang Liu - nano vLLM over TPU
3. Bo Chi, Krish Ydav - nano vLLM over MLX

Session 4: Quantization, Compression, and Sparse Attention
1. Benjamin Chauhan - Video Sparse Attention
2. Ethan Cheng, Anjaneya Gupta - Combining AWS and LLM.265

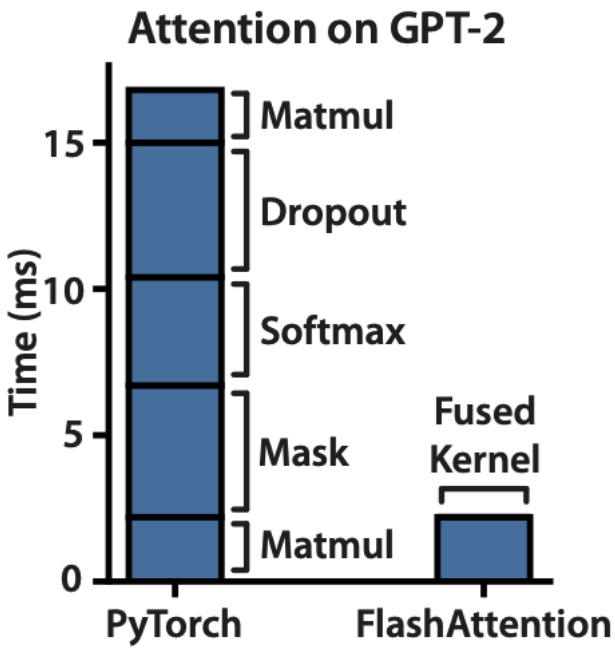
Session 5: Other interesting directions
1. Yifei Wang, Qinsi Wang - Agent Inference
2. Linda Wang - RL vs on-policy distillation
3. Akshar Yeccerla, Hung Le - Alpa in Phantora
4. Zanwen Fu - Spec decoding
5. Logan Chu - action conditioned world model



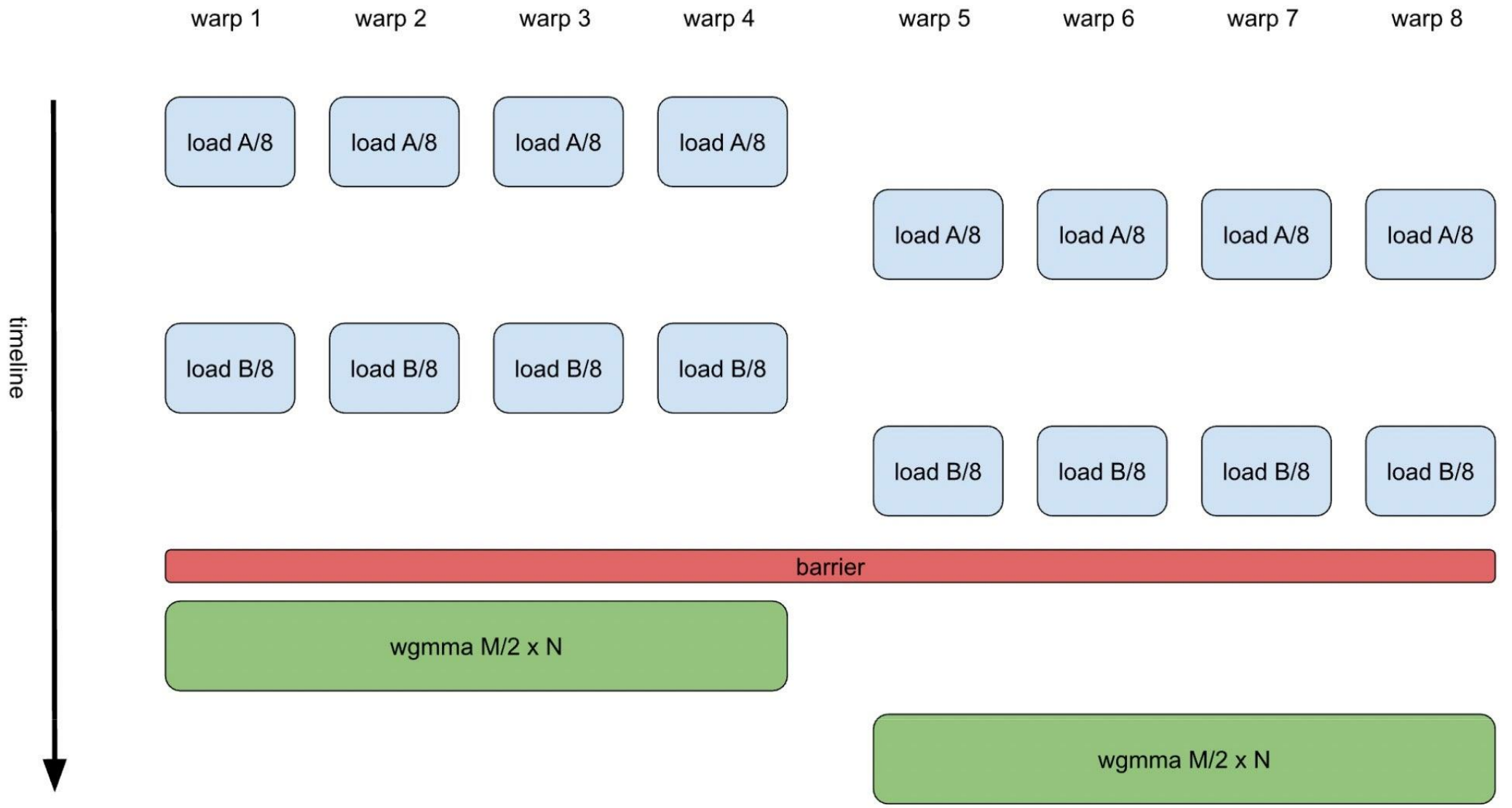
Memory Hierarchy with Bandwidth & Memory Size



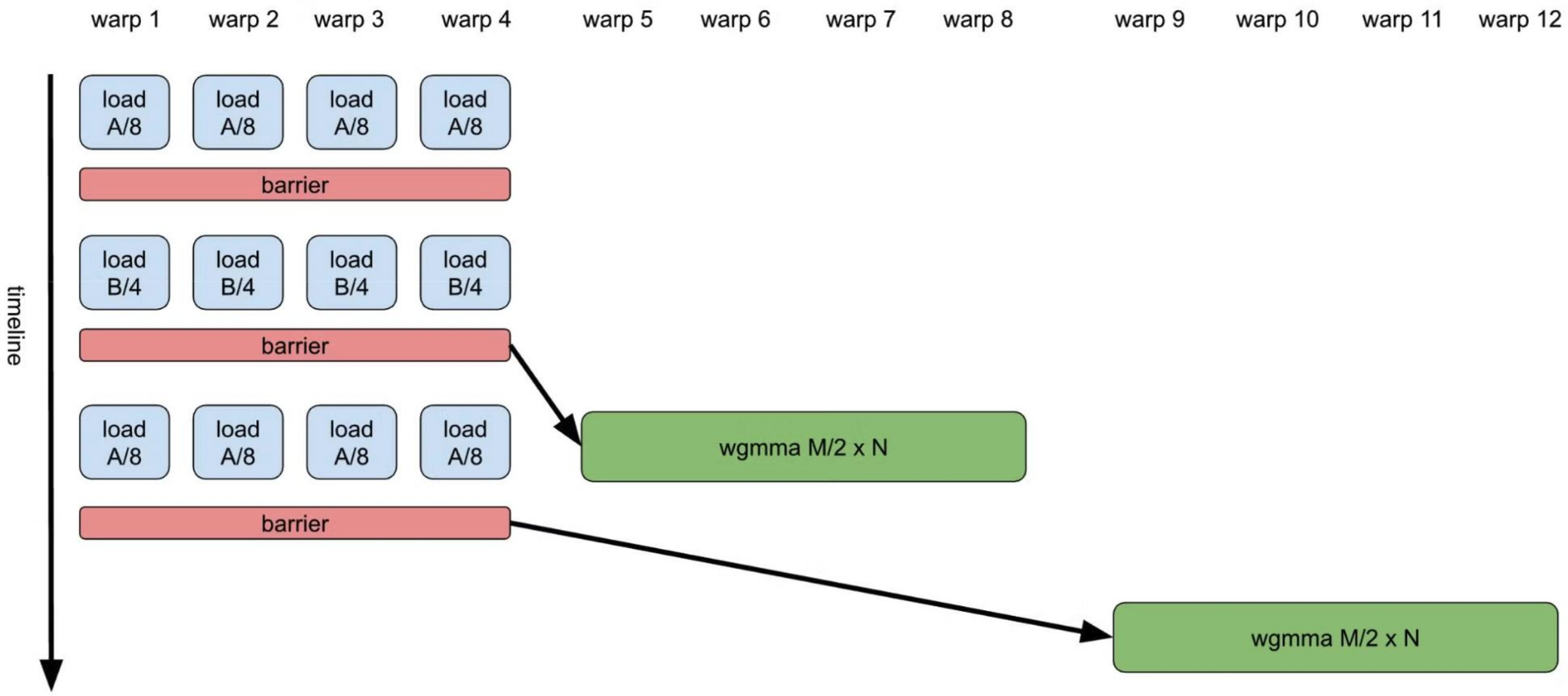
FlashAttention



FlashAttention


warp 1
warp 2
warp 3
warp 4
warp 5
warp 6
warp 7
warp 8
timeline
load A/8
load A/8
load A/8
load A/8
load A/8
load A/8
load A/8
load A/8
load B/8
load B/8
load B/8
load B/8
load B/8
load B/8
load B/8
load B/8
barrier
wgmma M/2 x N
wgmma M/2 x N

PyTorch


warp 1
warp 2
warp 3
warp 4
warp 5
warp 6
warp 7
warp 8
warp 9
warp 10
warp 11
warp 12
timeline
load A/8
load A/8
load A/8
load A/8
barrier
load B/4
load B/4
load B/4
load B/4
barrier
load A/8
load A/8
load A/8
load A/8
barrier
wgmma M/2 x N
wgmma M/2 x N

PyTorch

FlashAttention 3

- Hopper (H100, H200) started to have wrap specialization
  - TMA (tensor memory accelerator)
  - WGMMA (wrap-group matrix multiply accumulate)

- Technique #1: Overlapping of computation and data movement via wrap specialization
- Technique #2: Overlapping block matmul and softmax
- Technique #3: FP8

Algorithm 1 FlashAttention-3 forward pass **without** intra-consumer overlapping – CTA view

Require: Matrices $\mathbf{Q}_i \in \mathbb{R}^{B_r \times d}$ and $\mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, key block size B_c with $T_c = \lceil \frac{N}{B_c} \rceil$.

Initialize pipeline object to manage barrier synchronization with s-stage circular SMEM buffer.
if in producer warpgroup **then**
  Deallocate predetermined number of registers.
  Issue load $\mathbf{Q}_i$ from HBM to shared memory.
  Upon completion, commit to notify consumer of the load of $\mathbf{Q}_i$.
  for $0 \le j < T_c$ **do**
    Wait for the $(j \,\%\, s)$th stage of the buffer to be consumed.
    Issue loads of $\mathbf{K}_j, \mathbf{V}_j$ from HBM to shared memory at the $(j \,\%\, s)$th stage of the buffer.
    Upon completion, commit to notify consumers of the loads of $\mathbf{K}_j, \mathbf{V}_j$.
  end for
else
  Reallocate predetermined number of registers as function of number of consumer warps.
  On-chip, initialize $\mathbf{O}_i = (0) \in \mathbb{R}^{B_r \times d}$ and $\ell_i, m_i = (0), (-\infty) \in \mathbb{R}^{B_r}$.
  Wait for $\mathbf{Q}_i$ to be loaded in shared memory.
  for $0 \le j < T_c$ **do**
    Wait for $\mathbf{K}_j$ to be loaded in shared memory.
    Compute $\mathbf{S}_i^{(j)} = \mathbf{Q}_i \mathbf{K}_j^T$ (SS-GEMM). Commit and wait.
    Store $m_i^{\text{old}} = m_i$ and compute $m_i = \max(m_i^{\text{old}}, \text{rowmax}(\mathbf{S}_i^{(j)}))$.
    Compute $\widetilde{\mathbf{P}}_i^{(j)} = \exp(\mathbf{S}_i^{(j)} - m_i)$ and $\ell_i = \exp(m_i^{\text{old}} - m_i)\ell_i + \text{rowsum}(\widetilde{\mathbf{P}}_i^{(j)})$.
    Wait for $\mathbf{V}_j$ to be loaded in shared memory.
    Compute $\mathbf{O}_i = \text{diag}(\exp(m_i^{\text{old}} - m_i))^{-1}\mathbf{O}_i + \widetilde{\mathbf{P}}_i^{(j)}\mathbf{V}_j$ (RS-GEMM). Commit and wait.
    Release the $(j \,\%\, s)$th stage of the buffer for the producer.
  end for
  Compute $\mathbf{O}_i = \text{diag}(\ell_i)^{-1}\mathbf{O}_i$ and $L_i = m_i + \log(\ell_i)$.
  Write $\mathbf{O}_i$ and L_i to HBM as the ith block of $\mathbf{O}$ and L.
end if

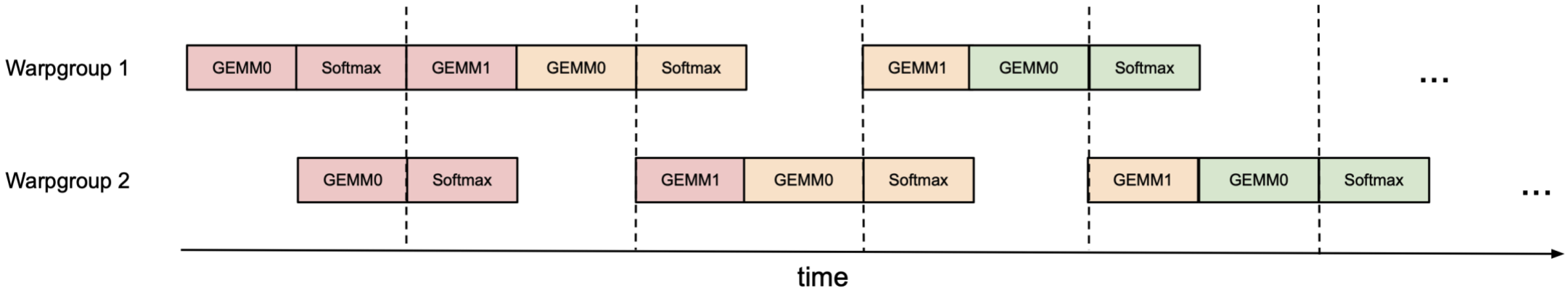


Figure 1: Pingpong scheduling for 2 warpgroups to overlap softmax and GEMMs: the softmax of one warpgroup should be scheduled when the GEMMs of another warpgroup are running. The same color denotes the same iteration.

Sliding Window Attention

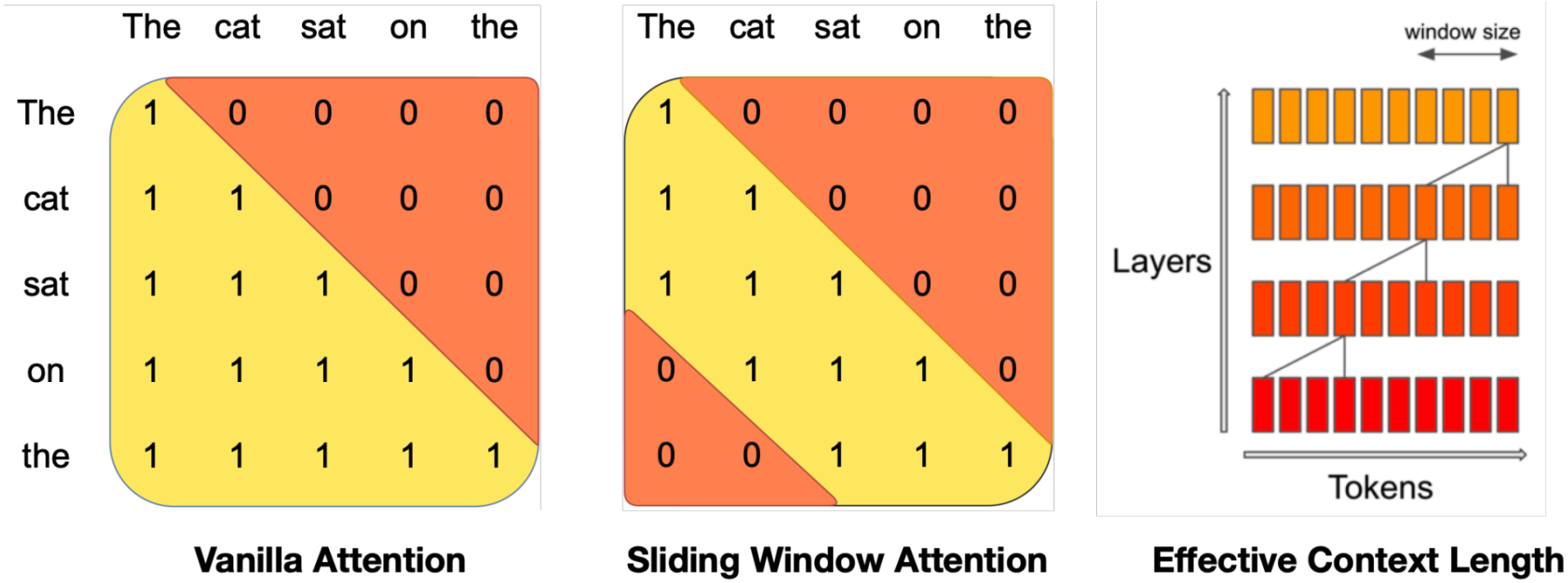


Figure 1: Sliding Window Attention. The number of operations in vanilla attention is quadratic in the sequence length, and the memory increases linearly with the number of tokens. At inference time, this incurs higher latency and smaller throughput due to reduced cache availability. To alleviate this issue, we use sliding window attention: each token can attend to at most W tokens from the previous layer (here, $W = 3$). Note that tokens outside the sliding window still influence next word prediction. At each attention layer, information can move forward by W tokens. Hence, after k attention layers, information can move forward by up to $k \times W$ tokens.

Mistral 7B

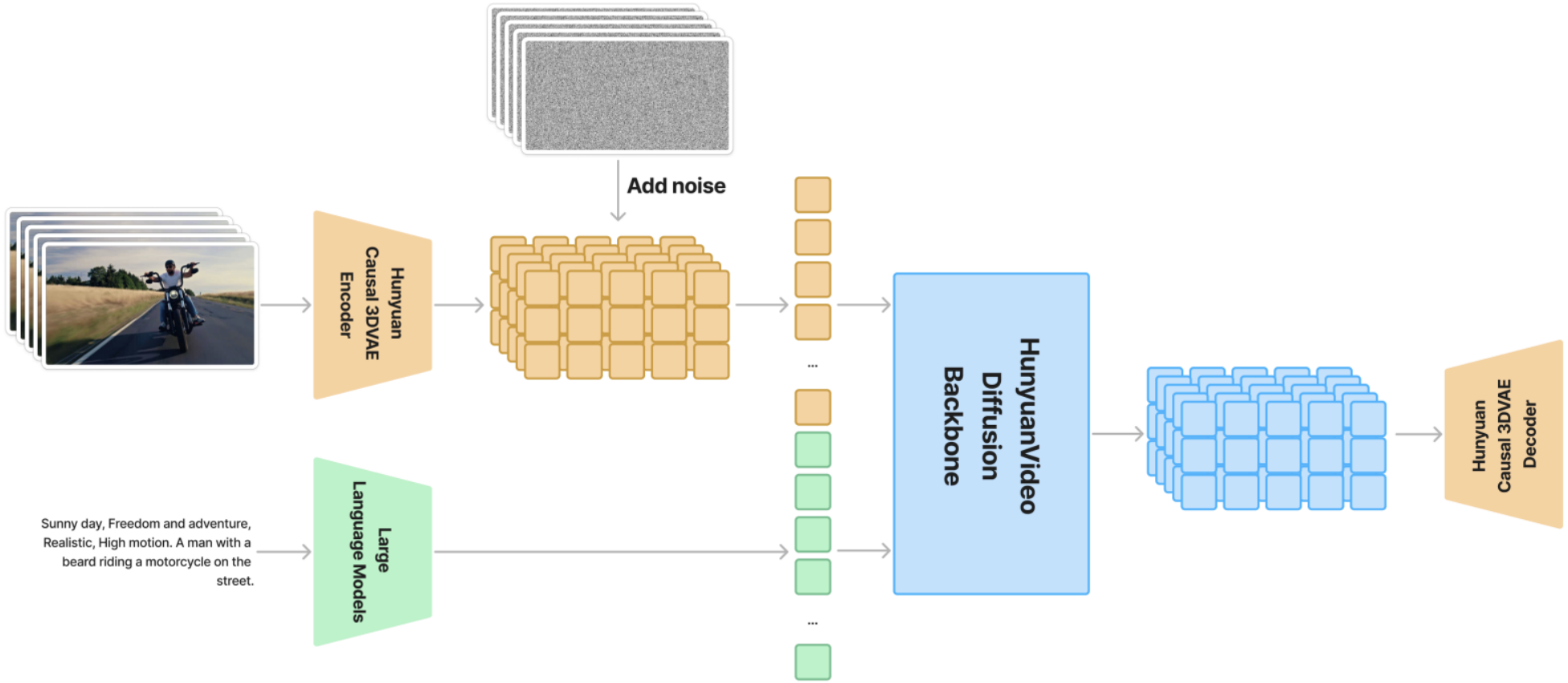
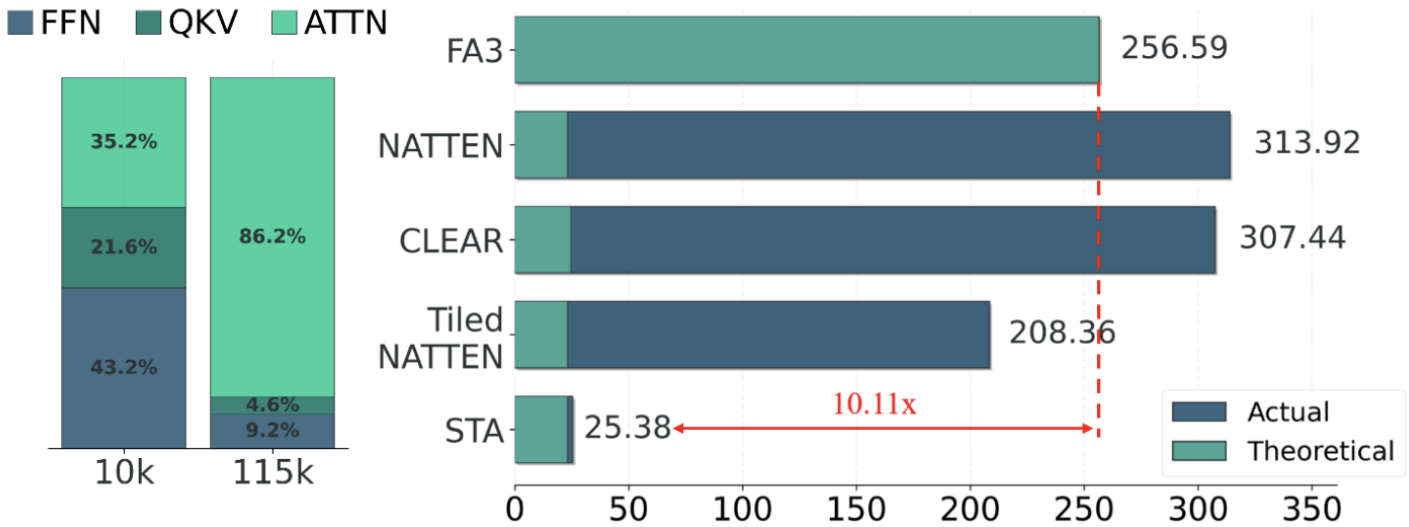


Figure 5: The overall architecture of HunyuanVideo. The model is trained on a spatial-temporally compressed latent space, which is compressed through Causal 3D VAE. Text prompts are encoded using a large language model, and used as the condition. Gaussian noise and condition are taken as input, our model generates a output latent, which is decoded into images or videos through the 3D VAE decoder.

HunyuanVideo

Video Generation is slow



(a) FLOPS distribution (b) Attention kernel latency(ms)

Figure 1. (a) Generating a 5s 720P clip in Hunyuan involves processing 115K tokens, making attention the dominant cost. (b) Attention latency comparison: existing methods fail to translate FLOP reduction into wall-clock speedup; STA is hardware-efficient and achieves proportional speedup with sparsity.

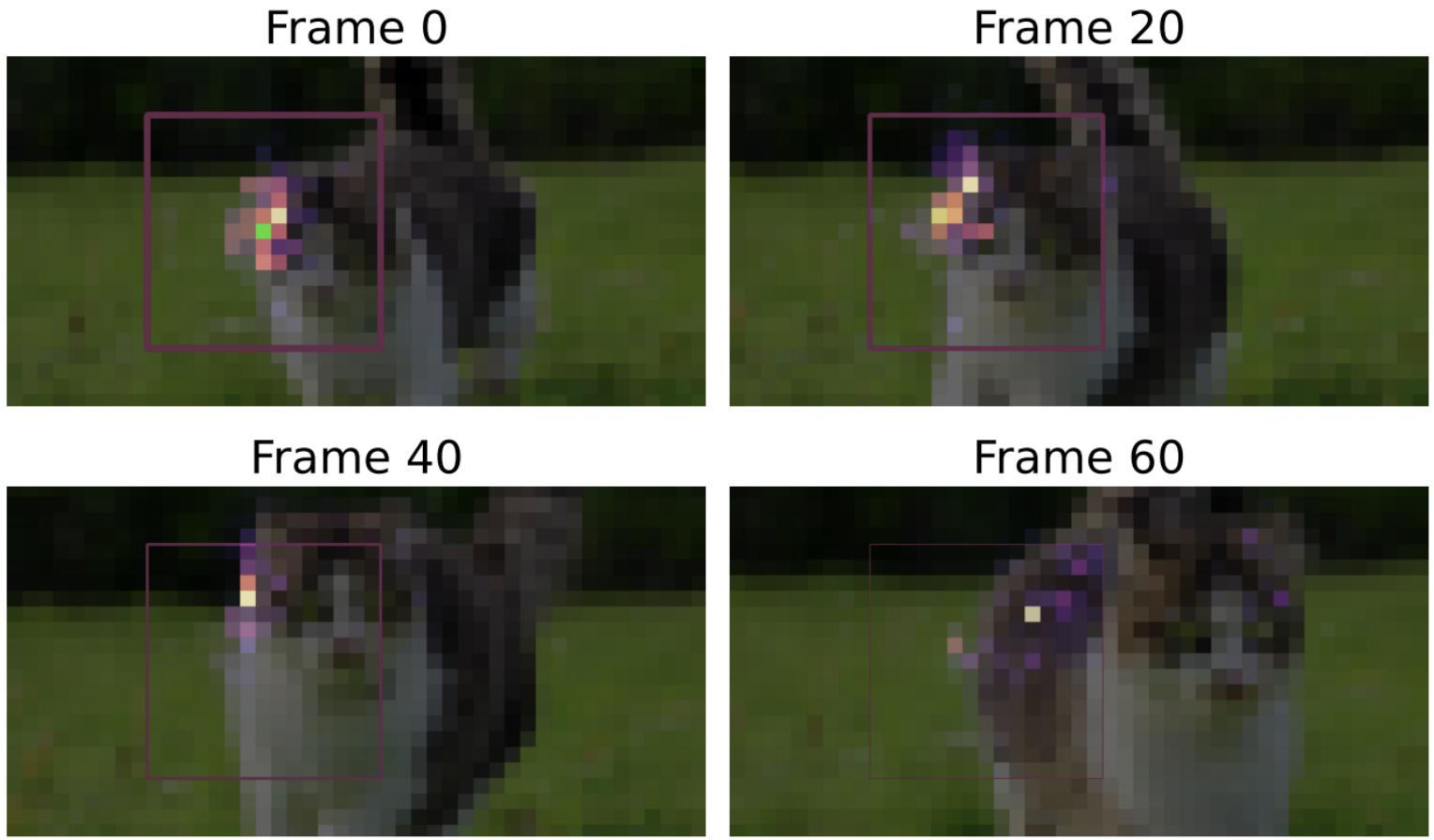


Figure 2. Visualization of attention locality. The green point means the query point and the magma-colored regions indicate areas of high attention values in response to the query. Instead of attending to the entire image, the query's attention forms a concentrated local hotspot.

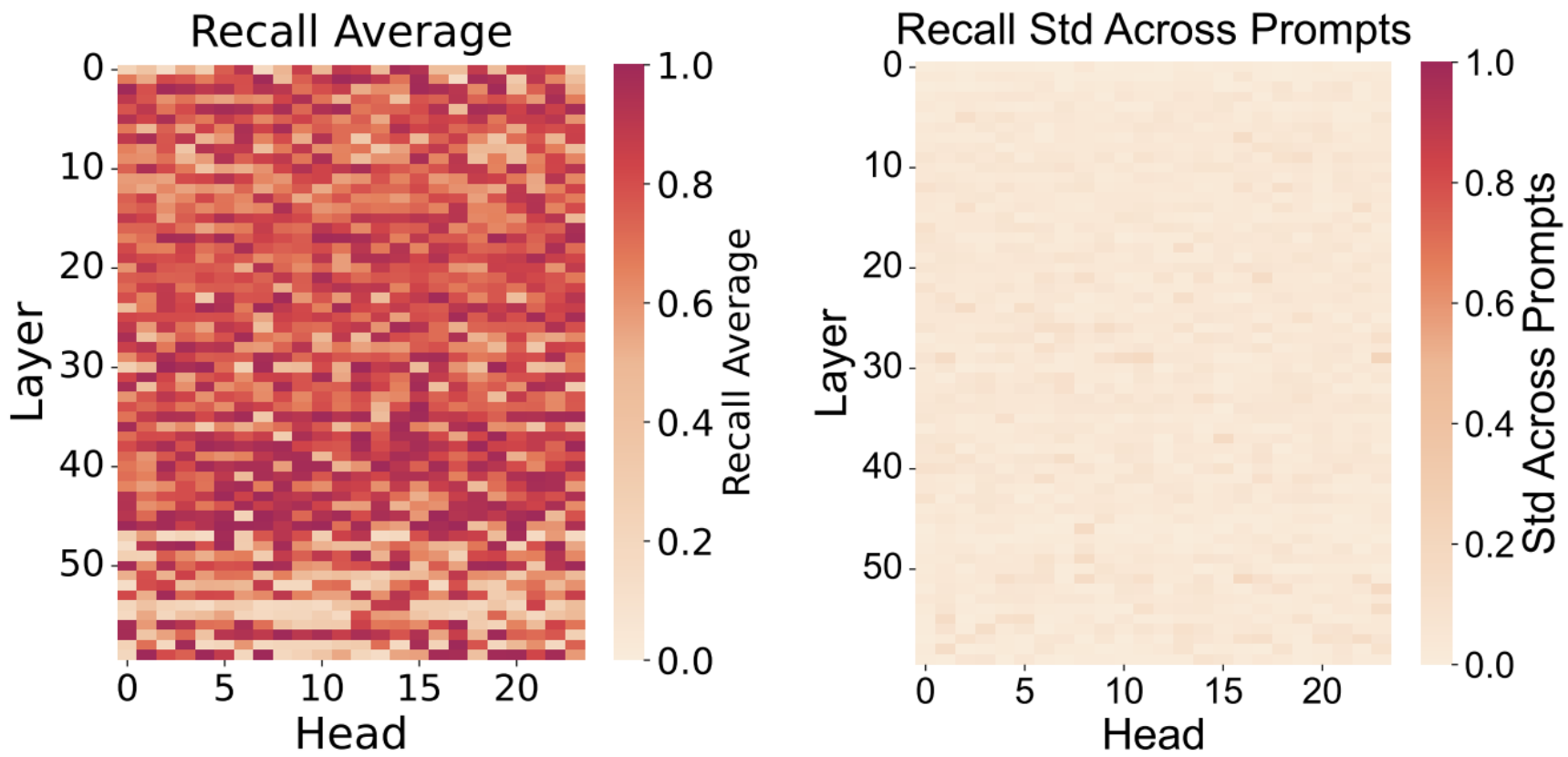


Figure 3. Left: Fraction of attention scores within a (12, 24, 24) local window across diffusion steps and 10 different prompts. Most heads show high recall, indicating a local attention pattern. *Right*: Despite the different recall across heads, the standard deviation across prompts remains low.

FastVideo

- Identify 3D locality and head specialization
- Sliding Tile Attention

Algorithm 1 STA Mask Search

Input: Transformer model M, Total steps T, Mask pattern list $\mathcal{P}$, Keep first T_0 timestep full attn

Output: Dictionary dict that stores selected mask pattern for each head

```
Initialize dict
for t = T_0 + 1 to T do
    for each layer head combination (l, h) in M do
        O ← (attn output of original (l, h) )
        Initialize minimum_loss ← ∞
        Initialize best_pattern ← null
        for each p in P do
            Mask head h for layer l using mask pattern p
            O' ← (attn output of M after masking)
            loss ← MSE(O, O')
            if loss < minimum_loss then
                minimum_loss ← loss
                best_pattern ← p
        Record best_pattern for (t, l, h) in dict
return dict
```

Sliding Window Attention

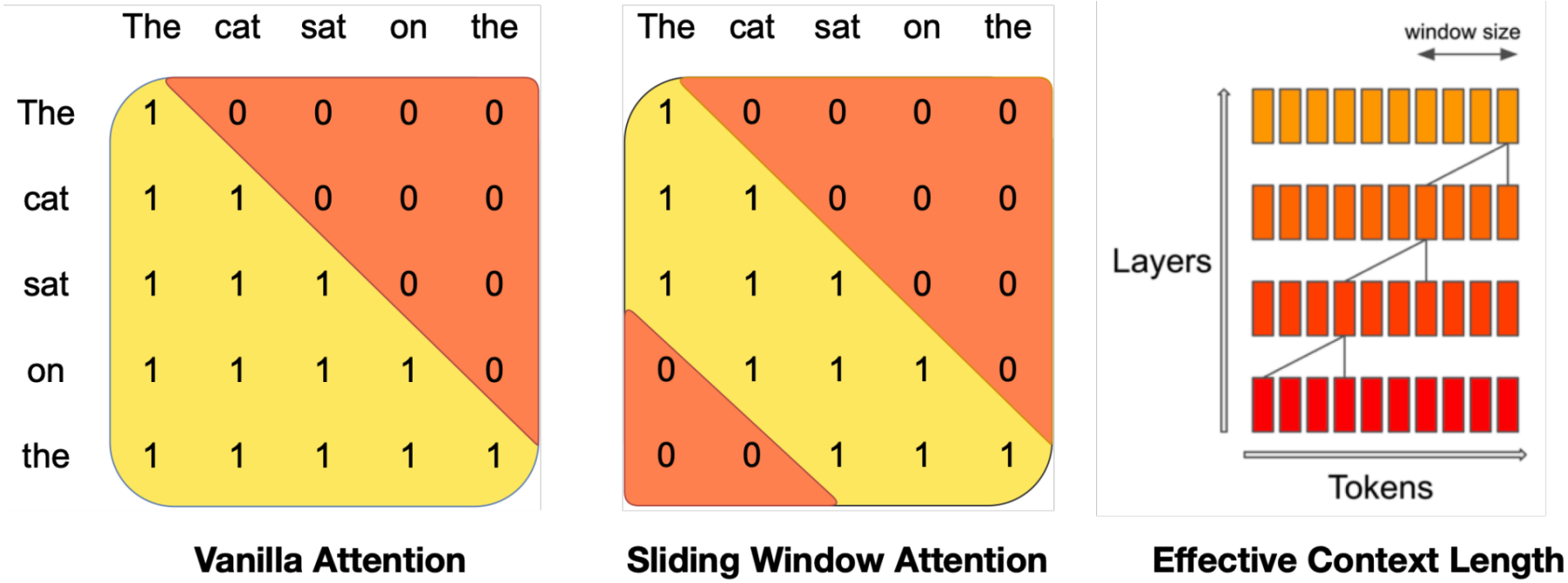


Figure 1: Sliding Window Attention. The number of operations in vanilla attention is quadratic in the sequence length, and the memory increases linearly with the number of tokens. At inference time, this incurs higher latency and smaller throughput due to reduced cache availability. To alleviate this issue, we use sliding window attention: each token can attend to at most W tokens from the previous layer (here, $W = 3$). Note that tokens outside the sliding window still influence next word prediction. At each attention layer, information can move forward by W tokens. Hence, after k attention layers, information can move forward by up to $k \times W$ tokens.

Mistral 7B

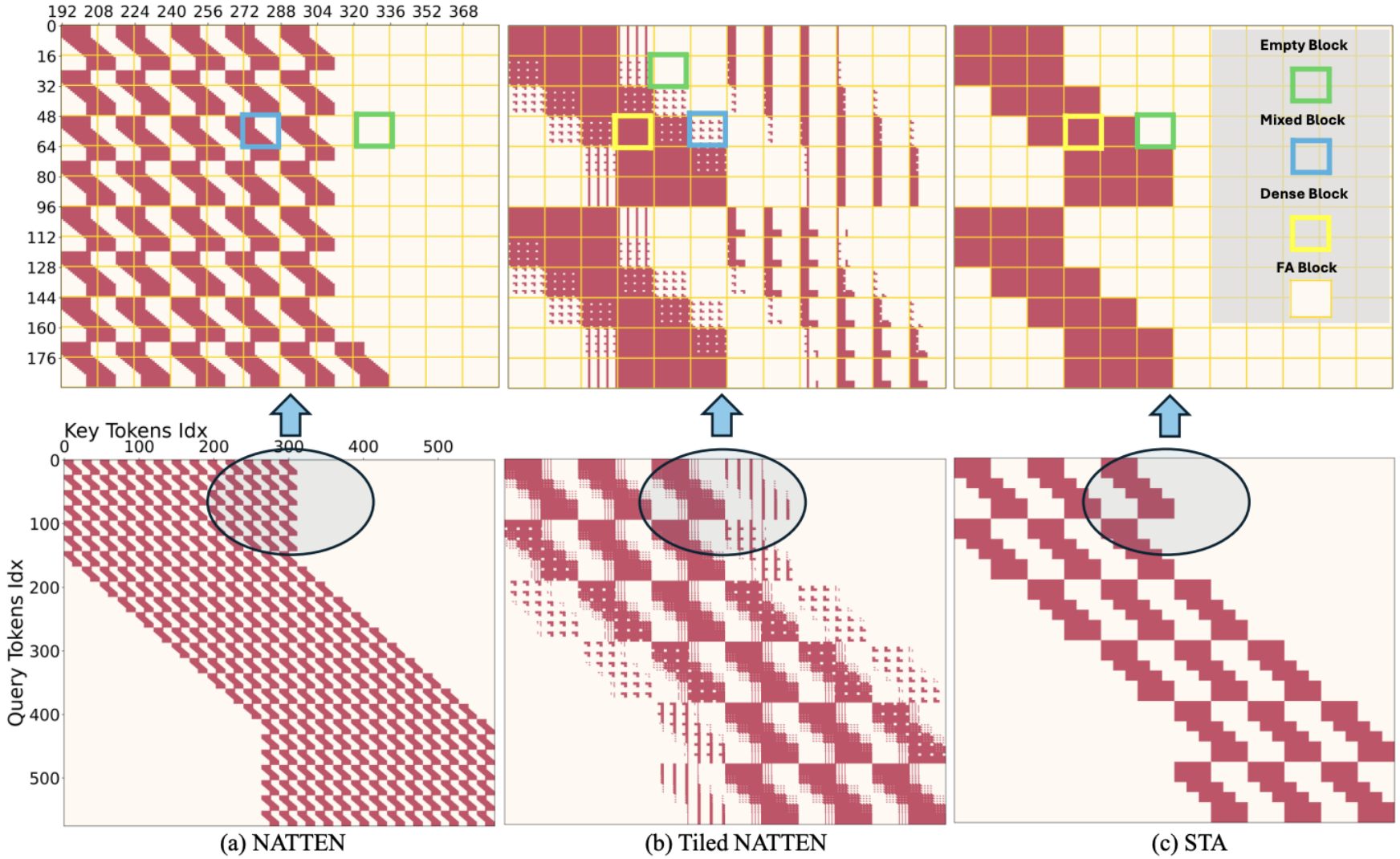


Figure 4. The attention map of NATTEN, Tiled NATTEN, and STA. We plot with an image size 24×24 and a 12×12 local window. The tile size is set to 4×4. (a) NATTEN creates many mixed blocks that are very inefficient for Flash Attention computation. (b) Tiled NATTEN increases the number of dense blocks, but the mixed blocks persist. (c) STA completely eliminates the mixed block, making the computation extremely friendly for GPU. Note that we mainly show STA's application in 3D scenarios for video generation in this paper, but for better illustration, we present the 2D scenario in this plot.

Sliding Tile Attention

- All queries in the same tile attend to the same set of keys within their common local window, ensuring a more structured attention pattern.

- By setting the tile area equal to the block size in FlashAttention and arranging queries in a tile with consecutive token indices, we form dense FlashAttention blocks.

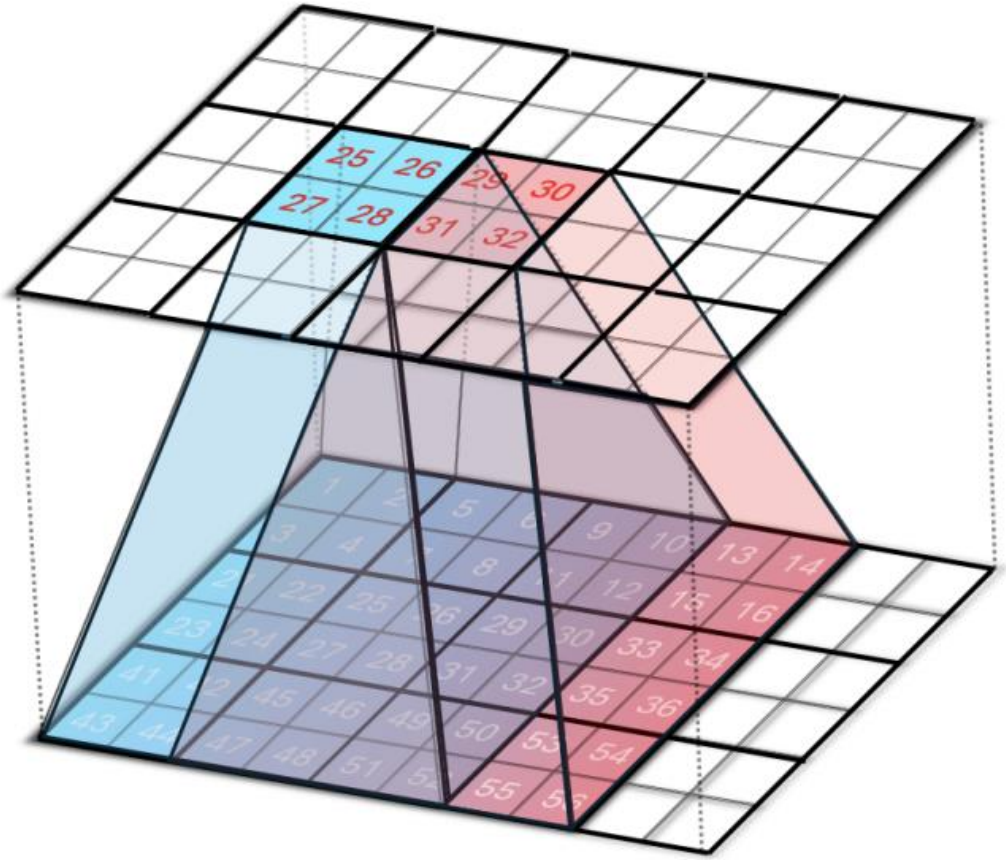


Figure 5. 2D SLIDING TILE ATTENTION with tile size (2, 2) and window size (6, 6). After attending to all the key tiles, each query tile will generate nine 4x4 dense blocks in the attention map. We showcase 2D STA for better illustration. 3D STA can be inferred similarly.

Fine-tuning

$$\mathcal{L}_{attn} = \frac{1}{N}\sum_{i=1}^{N} \| f_{\phi}^{(i)}(x_t, t, c) - f_{\psi}^{(i)}(x_t, t, c) \|_2^2,$$

$$\mathcal{L}_{final} = \| f_{\phi}(x_t, t, c) - f_{\psi}(x_t, t, c) \|_2^2$$

$$\mathcal{L}_{data} = \| (f - x_0) - f_{\phi}(x_t, t, c) \|_2^2,$$

$$\min_{\phi} \mathbb{E}_{x \sim p(x), c \sim N(0,1), t}[\alpha \mathcal{L}_{data} + \beta \mathcal{L}_{final} + \gamma \mathcal{L}_{attn}]$$

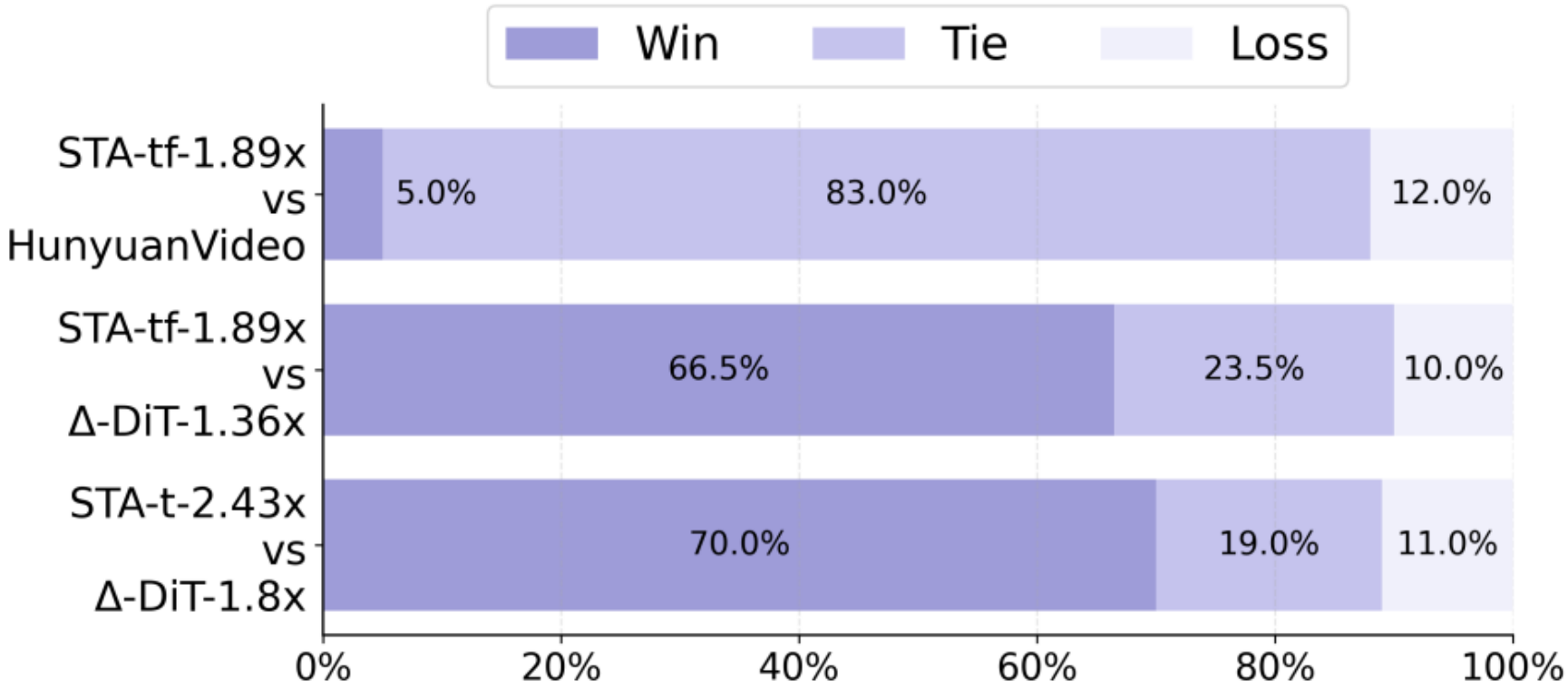


Figure 7. Human evaluation on 200 prompts from the MovieGen Bench (Polyak et al., 2024). STA achieves a 1.89× end-to-end speedup while maintaining performance comparable to the original HunyuanVideo. Additionally, STA consistently outperforms Δ-DiT across different inference budgets.

Table 4. Performance on VBench across different sparse attention patterns. STA achieves both high-quality video generation and significant speedup, while CLEAR and Tiled NATTEN suffer from efficiency issues and Swin suffers from quality degradation.

Methods	Config	VBench Quality	VBench Semantic	VBench Total	Attn Sparsity	PFLOPS	Latency	Speedup
FA2	–	85.34%	72.17%	82.71%	0.00%	574.16	1496s	0.63×
FA3	–	85.34%	72.17%	82.71%	0.00%	574.16	945s	1.00×
w.o training								
CLEAR	r=32	84.41%	74.20%	82.37%	56.23%	280.90	2567s	0.37×
Tiled NATTEN	w=(30,41,41)	84.61%	75.00%	82.69%	58.33%	269.92	1858s	0.51×
Swin	w=(48,64,64)	80.91%	71.35%	79.00%	55.81%	283.11	762s	1.24×
Swin	w=(30,40,40)	78.84%	72.28%	77.53%	76.49%	175.20	497s	1.90×
STA	w=(30,40,40)	84.63%	73.83%	82.46%	58.33%	269.92	527s	1.79×
STA	w=(18,24,24)	81.47%	77.03%	80.58%	91.00%	99.54	268s	3.53×
w. training								
Swin	w=(30,40,40)	77.50%	67.39%	75.48%	55.81%	283.08	497s	1.90×
STA	w=(30,24,40)	85.37%	73.52%	83.00%	75.00%	182.99	388s	2.44×
STA	w=(18,24,24)	84.76%	74.05%	82.62%	91.00%	99.54	268s	3.53×